Digital Egypt Pioneers Initiative — Data Analysis Track

TRAFFIC ACCIDENT ANALYTICS

Decoding the Roads — From Raw Crash Records to Public-Safety Insight

---

GRADUATION PROJECT DOCUMENTATION

2025 / 2026

Project Team

Ahmed Hossam Mohamed Nabil (Team Leader)
Fajr Abdullah
Youssef Amr Abdelhamid
Basmala Sabri
Eslam Jabr
Salma Adel

Table of Contents

Chapter 1 — Project Overview

Traffic collisions are among the most consistent and preventable sources of injury, fatality, and economic loss in urban environments. Every crash record filed by law enforcement carries a story about the interaction between road conditions, weather, human behavior, and the built environment. Analyzed at scale, these records can reveal exactly where, when, and under what conditions accidents cluster — information that is directly actionable for traffic engineering, policing, and public-safety policy.

1.1 Project Concept

Traffic Accident Analytics is a graduation project developed as part of the Digital Egypt Pioneers Initiative (DEPI), Data Analysis Track. The project analyzes a large, real-world crash dataset covering over 209,000 traffic accidents to uncover the environmental, behavioral, and structural conditions most associated with accident frequency and injury severity.

The analysis pipeline was built entirely in Python, following the Medallion Architecture (Bronze → Silver → Gold), and culminates in an interactive Power BI dashboard — first prototyped in Figma — designed to surface actionable insights for public-safety stakeholders, city planners, and the general public.

The motivation behind this project is twofold:

- Analytical: To apply the complete data analysis lifecycle — ingestion, cleaning, feature engineering, dimensional modeling, and interactive visualization — to a real-world, safety-critical domain.
- Social: To surface which road, weather, and time conditions carry the highest accident severity, so that resources (patrols, signage, road maintenance) can be targeted where they matter most.

Project objectives span four domains:

- Data Engineering: Ingest, validate, and clean a 209,306-row traffic accident dataset using Python (pandas / NumPy) across the Medallion Architecture layers.
- Feature Engineering: Derive meaningful fields — including Crash Severity, Time Buckets, and Damage Level — that go beyond the raw police-report fields to capture usable analytical categories.
- Dimensional Modeling: Design and populate a star-schema data model in the Gold layer, consisting of a central fact table and eight dimension tables optimized for Power BI consumption.
- Dashboard Development: Design (in Figma) and build (in Power BI) a multi-page interactive dashboard covering accident overview, road conditions, weather impact, severity, and crash causes, backed by a full set of DAX measures.
- Predictive Analytics: Forecast future monthly accident volume and injury rates, and model how crash severity and damage level relate to weather and road surface, using XGBoost and Prophet time-series models (see Chapter 5, AI Insights).

1.2 Data Source

The dataset used in this project is the Traffic Accidents dataset, published on Kaggle by the user oktayrdeki. It contains real-world, police-reported traffic crash records, each row representing a single crash event along with its environmental context, contributing cause, and injury outcome.

Attribute	Detail
Dataset Name	Traffic Accidents
Source Platform	Kaggle
Publisher	oktayrdeki
Dataset URL	kaggle.com/datasets/oktayrdeki/traffic-accidents
Data Type	Real-world, police-reported crash records
Total Rows	209,306 records
Rows Used in Project	209,306 records (full dataset)
Date Range	March 2013 – January 2025 (≈11.9 years; 2013 and 2025 are partial years)
File Format	CSV
License	Open / Public (Kaggle)

The raw dataset contains 24 columns spanning four thematic domains: crash context (date, control device, weather, lighting, road type), crash classification (type, severity, contributory cause), injury outcomes, and time attributes.

Column Name	Type	Description
crash_date	Datetime	Date and time the crash occurred
traffic_control_device	String	Traffic control device present at the crash site (signal, stop sign, none, etc.)
weather_condition	String	Weather condition at the time of the crash
lighting_condition	String	Lighting condition at the time of the crash (daylight, darkness, dusk, dawn)
first_crash_type	String	Type of the initial collision (rear end, angle, sideswipe, pedestrian, etc.)
trafficway_type	String	Type of trafficway where the crash occurred (intersection type, one-way, divided, etc.)
alignment	String	Roadway alignment at the crash location (straight/level, curve, hillcrest)
roadway_surface_cond	String	Condition of the road surface (dry, wet, snow/slush, ice, unknown)
road_defect	String	Roadway defect present, if any (none, rut/holes, worn surface, debris)
crash_type	String	High-level crash outcome classification (injury/tow vs. no-injury/drive-away)
intersection_related_i	String	Whether the crash was related to an intersection (Y / N)
damage	String	Estimated cost-of-damage bracket ($500 or less, $501–$1,500, over $1,500)
prim_contributory_cause	String	Primary contributory cause assigned to the crash
num_units	Integer	Number of vehicles / units involved in the crash
most_severe_injury	String	Most severe injury classification reported in the crash
injuries_total	Float	Total number of injuries recorded in the crash
injuries_fatal	Float	Number of fatal injuries in the crash

Column Name	Type	Description
injuries_incapacitating	Float	Number of incapacitating (severe) injuries
injuries_non_incapacitating	Float	Number of non-incapacitating (evident, non-severe) injuries
injuries_reported_not_evident	Float	Number of injuries reported but not visibly evident at the scene
injuries_no_indication	Float	Number of persons with no indication of injury
crash_hour	Integer	Hour of day the crash occurred (0–23)
crash_day_of_week	Integer	Day of week the crash occurred (1–7)
crash_month	Integer	Month the crash occurred (1–12)

1.3 Project Impact (KPIs & Measures)

The project produces a set of engineered KPIs and DAX measures that go beyond the raw dataset fields, giving stakeholders a direct read on accident volume, injury severity, and risk concentration. These values were first validated in the Python EDA notebook (Chapter 4) and are reproduced as DAX measures in Power BI (Chapter 6).

Accident Volume KPIs

KPI	Value	Derivation
Total Accidents	209,306	COUNTROWS of fact_accidents
Total Vehicles Involved	431,861	SUM of Number_of_vehicles_involved
Avg. Vehicles per Accident	2.06	Total Vehicles Involved / Total Accidents
Date Range Covered	Mar 2013 – Jan 2025 (~11.9 yrs)	MAX(Date) − MIN(Date) in dim_date

Injury & Severity KPIs

KPI	Value	Derivation
Total Injuries	80,105	SUM of Total_injuries_num
Avg. Injuries per Accident	0.38	Total Injuries / Total Accidents
Total Fatal Injuries	389	SUM of Fatal_injuries_num
Incapacitating Injuries	7,975	SUM of injuries_incapacitating
Non-Incapacitating Injuries	46,307	SUM of injuries_non_incapacitating
Not Visibly Evident Injuries	25,434	SUM of injuries_not_visibily_evident
Severe Injuries	8,364	Fatal Injuries + Incapacitating Injuries
Fatal Injury Rate	0.49%	Fatal Injuries / Total Injuries
Incapacitating Injury Rate	9.96%	Incapacitating Injuries / Total Injuries
Severity Rate	10.44%	Severe Injuries / Total Injuries
Severity Index	0.6857	Weighted injury score / Total Accidents (see 4.2)

Road, Weather & Damage KPIs

KPI	Value	Derivation
Accidents on Clear Weather	164,700 (78.7%)	COUNT where Weather = Clear
Accidents on Wet / Icy / Snow Roads	40,414 (19.3%)	COUNT where Road_Surface ∈ {Wet, Ice, Snow/Slush}
Accidents in Daylight	134,109 (64.1%)	COUNT where Lighting = Daylight
Accidents in Darkness (any)	60,814 (29.1%)	COUNT where Lighting ∈ {Darkness, Darkness-Lighted Road}
High-Damage Accidents (>$1,500)	147,313 (70.4%)	COUNT where Damage_level = 3
Injury/Tow Crash Type Share	91,930 (43.9%)	COUNT where Crash_Type = Injury and/or Tow

1.4 Main Charts Overview

The Power BI dashboard is structured as a data storytelling journey — beginning with a landing/portal page, then a high-level overview, before drilling into severity, road conditions, crash causes, and weather. This structure mirrors the Figma prototype described in section 4.3.

Page	Title	Focus Area	Key Chart Types
0	Public Safety Portal	Entry point / navigation	Hero image, navigation buttons
1	Dashboard Overview	High-level accident snapshot	KPI cards, line chart, donut chart, summary tables
2	Severity Analysis	Injury severity patterns	KPI cards, heatmap grid, trend chart, bar chart
3	Road Condition Analysis	Road surface & defect impact	KPI cards, bar charts, treemap
4	Crash Cause Analysis	Contributory causes	KPI cards, table, stacked bar, column chart
5	Weather Condition Analysis	Weather impact on accidents & injuries	KPI cards, bar charts, stacked bar

1.5 SMART Questions

The analysis is guided by five SMART (Specific, Measurable, Achievable, Relevant, Time-bound) research questions derived from the dataset structure and project objectives:

#	SMART Question	KPI / Measure
1	Across the 209,306 accidents recorded between 2013 and 2025, does the Severity Index vary meaningfully by lighting condition, and are darkness-related crashes measurably more severe than daylight crashes?	Severity Index by Lighting
2	What percentage of accidents occur on wet, icy, or snow/slush road surfaces, and do these conditions produce a higher Severity Rate than accidents on dry roads?	Severity Rate, Road Surface %
3	Do specific days of week (e.g. Friday, Thursday, Saturday) and specific months (e.g. October, September, December) show accident volumes significantly above the yearly average, indicating weekly or seasonal risk windows?	Total Accidents by Day / Month

#	SMART Question	KPI / Measure
4	Is there a measurable relationship between the estimated cost-of-damage bracket and injury severity — do high-damage crashes (>$1,500) also carry a higher share of incapacitating and fatal injuries?	Damage_level, Severe Injuries
5	Which primary contributory causes account for the largest share of accidents, and do the top causes differ between injury/tow crashes and no-injury/drive-away crashes?	Total Accidents by Contributory Cause, Crash Type

Chapter 2 — Technical Architecture

This chapter documents the tools, architectural pattern, and data model that form the technical backbone of the project. The pipeline was built entirely in Python (pandas / NumPy), following the Medallion Architecture, and outputs a star-schema dimensional model consumed by Power BI.

2.1 Tools Used

Four primary tools were used across the full pipeline — from raw data ingestion and cleaning, through exploratory analysis and dimensional modeling, to dashboard design and delivery:

Tool / Platform	Role in Project	Key Usage
Python (Bronze/Silver/Gold notebooks)	Data engineering & Medallion pipeline	Ingesting the raw CSV from Kaggle, cleaning and standardizing fields, engineering derived columns, and building the star-schema dimension and fact tables using pandas and NumPy.
Python (EDA notebook)	Exploratory data analysis	Data-quality checks, distribution analysis, ranking queries, and construction of the full measures dictionary later reproduced as DAX in Power BI; charts built with Plotly Express.
Figma	UI/UX design & dashboard wireframing	Used to design and prototype the six-page dashboard layout, color scheme, navigation structure, and visual hierarchy — including the 'Vigilant Analytics' brand system — before implementation in Power BI.
Power BI	Business intelligence & interactive dashboard	Consumed the Gold layer tables to build the multi-page 'Traffic Accident Analytics' dashboard, including KPI cards, charts, heatmaps, and slicers, driven by a full DAX measure library.

The Bronze, Silver, and Gold notebooks were run sequentially in Python: the raw dataset was read directly from CSV into a pandas DataFrame, cleaned and enriched in the Silver stage, then split into a star schema of one fact table and eight dimension tables in the Gold stage, each exported as CSV for downstream consumption by the EDA notebook and Power BI.

Figma was used during the dashboard design phase to plan the page structure, navigation, visual hierarchy, and color palette before a single visual was placed in Power BI. This design-first approach allowed the team to align on the storytelling flow — from a public-facing landing page through to weather-condition analysis — and reduce iteration time during Power BI development.

2.2 Architecture (Medallion)

The Medallion Architecture organizes the pipeline into three logical layers — Bronze, Silver, and Gold — each serving a specific purpose. This ensures the raw data is never overwritten (Bronze), all cleaning and enrichment logic is centralized (Silver), and the analytical layer (Gold) is purpose-built for Power BI performance.

Layer	Purpose	Key Operations	Output
Bronze	Raw ingestion	Read raw CSV from Kaggle into pandas; inspect schema with .info() and .describe(); no transformation applied	Bronze.csv (209,306 rows × 24 columns)
Silver	Cleaning & feature engineering	Type conversion, value standardization, column renaming, derived columns (Crash_severity, Time Buckets, Damage_level)	Silver.csv (enriched, same row count)
Gold	Star schema build	Split into 8 dimension tables + 1 fact table, join via surrogate keys	fact_accidents.csv + 8 dim_*.csv files

2.3 Data Model (Star Schema)

The Gold layer implements a star schema consisting of one central fact table surrounded by eight dimension tables, each connected via a surrogate key generated during the Gold-layer build. All tables are exported as flat CSV files for direct Power BI import.

Table	Type	Key Column(s)	Description
fact_accidents	Fact	Accident_Key + 7 FKs	Central table: one row per crash, holding all foreign keys plus numeric injury and vehicle-count measures
dim_date	Dimension	Date_Key	Calendar attributes: Date, Year, Month, Month_Name, Day, Quarter, Day_Of_Week
dim_weather	Dimension	Weather_Key	Distinct weather conditions at the time of the crash
dim_lighting	Dimension	Lighting_Key	Distinct lighting conditions at the time of the crash
dim_crash_type	Dimension	Crash_Type_Key	Injury/Tow vs. No-Injury/Drive-Away crash outcome classification
dim_road	Dimension	Road_Key	Distinct roadway surface conditions
dim_time	Dimension	Time_Key	Crash hour, AM/PM flag, and the engineered Time Buckets category
dim_damage	Dimension	Damage_Key	Cost-of-damage bracket paired with the engineered Damage_level (1–3)
dim_severity	Dimension	Severity_Key	Engineered Crash_severity category derived from injury thresholds

Every dimension is joined to fact_accidents on its surrogate key, giving the model a clean, single-hop star shape optimized for the slicer-heavy, drill-down interactions used throughout the Power BI dashboard.

Chapter 3 — Data Engineering & Analysis

This chapter documents the complete data preparation and transformation pipeline across the three Medallion Architecture layers, implemented in Python using pandas and NumPy, plus the exploratory analysis that produced the project's headline measures.

3.1 Cleaning Process

The cleaning process spans the Bronze and Silver layers. Bronze ingests and preserves the raw data exactly as downloaded; Silver applies all type corrections, value standardization, and enrichment.

Bronze Layer — Raw Ingestion

Source: Bronze_layer.ipynb

The Bronze layer reads the raw CSV from Kaggle into a pandas DataFrame and immediately profiles it with .info() and .describe() before any transformation is applied.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

pd.options.display.float_format = '{:,.2f}'.format
pd.options.display.max_rows = None
pd.options.display.max_columns = None

# Load the raw dataset into the Bronze DataFrame
Bronze_df = pd.read_csv("/content/sample_data/traffic_accidents.csv")
Bronze_df.info()
Bronze_df.describe()
```

Profiling confirmed 209,306 rows across 24 columns with no missing values in any field. Numeric fields such as injuries_total (max 21), injuries_fatal (max 3), and num_units (max 11) were inspected for plausibility before being carried into the Silver layer unchanged.

```
# Persist the untouched raw data as the permanent Bronze archive
Bronze_df.to_csv(
    "/content/sample_data/Bronze.csv",
    index=False
)
print("Bronze Layer Created Successfully")
```

Silver Layer — Data Cleaning & Standardization

Source: Silver_Layer.ipynb

The Silver layer reads the Bronze CSV and applies twelve sequential cleaning and enrichment steps. No rows are dropped — every operation either corrects a data type, standardizes a value, or adds a derived column, preserving the full 209,306-row dataset.

1. Type correction — crash_date is parsed to a real datetime, and all categorical text columns are cast to string to guard against mixed-type inference:

```
Silver_df["crash_date"] = pd.to_datetime(
    Silver_df["crash_date"],
    errors="coerce"
)

text_cols = [
    "traffic_control_device", "weather_condition", "lighting_condition",
    "first_crash_type", "trafficway_type", "alignment",
    "roadway_surface_cond", "road_defect", "crash_type",
    "intersection_related_i", "damage", "prim_contributory_cause",
    "most_severe_injury"
]
for col in text_cols:
    Silver_df[col] = Silver_df[col].astype(str)
```

2. Value standardization — the shorthand Y/N codes in intersection_related_i are replaced with readable labels:

```
Silver_df["intersection_related_i"] = (
    Silver_df["intersection_related_i"]
    .replace({"Y": "YES", "N": "NO"})
)
```

3–5. Time extraction — crash_date is duplicated, the AM/PM period is extracted from the timestamp, and the original column is reduced to a pure date:

```
Silver_df["AM/PM"] = Silver_df["crash_date"]
Silver_df["AM/PM"] = (
    pd.to_datetime(Silver_df["AM/PM"])
    .dt.strftime("%p")
)
Silver_df["crash_date"] = Silver_df["crash_date"].dt.date
```

6. Column renaming — raw police-report field names are renamed to clear, dashboard-ready labels:

```
Silver_df = Silver_df.rename(columns={
    "intersection_related_i": "Intersection_accident?",
    "damage":                 "Cost_of_damage",
    "weather_condition":      "Weather",
    "lighting_condition":     "Lighting",
    "first_crash_type":       "Initial_crash_type",
    "alignment":              "Road_alignment",
    "num_units":              "Number_of_vehicles_involved",
    "injuries_total":         "Total_injuries_num",
    "injuries_fatal":         "Fatal_injuries_num",
    "injuries_reported_not_evident": "injuries_not_visibily_evident"
})
```

10. Data-inconsistency handling — the redundant 'UNKNOWN INTERSECTION TYPE' label in trafficway_type is collapsed into the standard 'UNKNOWN' value:

```
Silver_df["trafficway_type"] = (
    Silver_df["trafficway_type"]
    .replace({"UNKNOWN INTERSECTION TYPE": "UNKNOWN"})
)
```

12. Final type correction — crash_date is restored to a proper datetime dtype (it had reverted to an object/string after the .dt.date step) before being written to disk:

```
Silver_df['crash_date'] = pd.to_datetime(Silver_df['crash_date'])
```

Source: Silver_Layer.ipynb

```
Silver_df.to_csv(

"/content/sample_data/Silver.csv",
    index=False
)
print("Silver Layer Created
Successfully")
```

Step	Issue / Task	Correction Applied
1	crash_date stored as text; categorical columns risk mixed dtypes	Parsed to datetime; text columns cast to string
2	Y/N shorthand in intersection flag	Replaced with YES / NO
3–5	Time-of-day information embedded in the timestamp	Extracted AM/PM period; reduced crash_date to date-only
6	Raw police-report column names are not analysis-friendly	Renamed 10 columns to clear business labels
7	No single severity indicator existed	Engineered Crash_severity (see 3.2)
8	Column order was inconsistent with reporting needs	Reordered all 26 columns into a logical sequence
9	crash_hour alone is hard to group by daypart	Engineered Time Buckets (see 3.2)
10	Duplicate label variant in trafficway_type	Standardized 'UNKNOWN INTERSECTION TYPE' → 'UNKNOWN'
11	Cost_of_damage bracket had no numeric severity proxy	Engineered Damage_level 1–3 (see 3.2)
12	crash_date reverted to object dtype after .dt.date	Re-cast to datetime before export

3.2 Feature Engineering

Source: Silver_Layer.ipynb

Three derived columns were engineered from the cleaned raw fields, capturing analytical categories that are not directly observable from any single raw column.

Crash Severity

A categorical severity label is derived from the injury-count columns using a priority cascade — the first true condition (checked top to bottom) wins, from most to least severe:

```
Silver_df["Crash_severity"] = np.select(
    [
        Silver_df["Fatal_injuries_num"] > 0,
        Silver_df["injuries_incapacitating"] > 0,
        Silver_df["injuries_non_incapacitating"] > 0,
        Silver_df["Total_injuries_num"] > 0
    ],
    ["Fatal", "Major", "Moderate", "Minor"],
    default="No Injury"
)
```

Time Buckets

crash_hour is bucketed into five human-readable dayparts, used throughout the dashboard for time-of-day risk analysis:

```
conditions = [
    Silver_df["crash_hour"] <= 5,
    (Silver_df["crash_hour"] >= 6)  & (Silver_df["crash_hour"] <= 11),
    (Silver_df["crash_hour"] >= 12) & (Silver_df["crash_hour"] <= 17),
    (Silver_df["crash_hour"] >= 18) & (Silver_df["crash_hour"] <= 21),
    (Silver_df["crash_hour"] >= 22) & (Silver_df["crash_hour"] <= 23),
]
values = ["Late Night", "Morning", "Afternoon", "Evening", "Night"]

Silver_df["Time Buckets"] = np.select(conditions, values, default=None)
```

Damage Level

The text-based Cost_of_damage bracket is mapped to a numeric 1–3 severity proxy, making it usable in numeric aggregations and conditional formatting:

```
def damage_level(row):
    if row['Cost_of_damage']
== 'OVER $1,500':
        return 3
    elif
row['Cost_of_damage'] ==
'$501 - $1,500':
        return 2
    elif
row['Cost_of_damage'] ==
'$500 OR LESS':
        return 1
    else:
        return 'unknown'

Silver_df['Damage_level'] =
Silver_df.apply(damage_level,
axis=1)
```

Engineered Column	Type	Values	Purpose
Crash_severity	Categorical	Fatal / Major / Moderate / Minor / No Injury	Single injury-severity label for filtering and severity KPIs

Time Buckets	Categorical	Late Night, Morning, Afternoon, Evening, Night	Daypart grouping for time-of-day accident and risk analysis
Damage_level	Numeric (1–3)	1 = ≤$500, 2 = $501–$1,500, 3 = >$1,500	Numeric proxy for the cost-of-damage bracket
AM/PM	Categorical	AM / PM	Half-day period extracted from the crash timestamp

Gold Layer — Star Schema Build

Source: Gold_Layer.ipynb

The Gold layer reads the enriched Silver DataFrame and splits it into eight dimension tables plus one fact table, generating a surrogate key for each dimension and joining those keys back onto the fact rows.

```
Gold_df = pd.read_csv("/content/sample_data/Silver.csv")
Gold_df["crash_date"] = pd.to_datetime(Gold_df["crash_date"])

# --- dim_date ---
dim_date = pd.DataFrame()
dim_date["Date"] = pd.to_datetime(Gold_df["crash_date"].unique())
dim_date = dim_date.sort_values("Date")
dim_date["Date_Key"] = range(1, len(dim_date) + 1)
dim_date["Year"] = dim_date["Date"].dt.year
dim_date["Month"] = dim_date["Date"].dt.month
dim_date["Month_Name"] = dim_date["Date"].dt.month_name()
dim_date["Day"] = dim_date["Date"].dt.day
dim_date["Quarter"] = dim_date["Date"].dt.quarter
dim_date["Day_Of_Week"] = dim_date["Date"].dt.day_name()
```

The remaining seven dimensions (dim_weather, dim_lighting, dim_crash_type, dim_road, dim_time, dim_damage, dim_severity) follow the same pattern — extract distinct values from the relevant Silver column(s) and assign a sequential surrogate key:

```
dim_weather = pd.DataFrame({"Weather": Gold_df["Weather"].dropna().unique()})
dim_weather["Weather_Key"] = range(1, len(dim_weather) + 1)

dim_time = Gold_df[["crash_hour", "AM/PM", "Time Buckets"]].drop_duplicates()
dim_time["Time_Key"] = range(1, len(dim_time) + 1)

dim_damage = Gold_df[["Cost_of_damage", "Damage_level"]].drop_duplicates()
dim_damage["Damage_Key"] = range(1, len(dim_damage) + 1)
```

The fact table is assembled by left-joining each dimension's surrogate key onto the full Silver dataset, then keeping only the keys and the numeric measure columns:

```
fact = Gold_df.copy()
fact = fact.merge(dim_date[["Date_Key", "Date"]], left_on="crash_date", right_on="Date",
how="left")
fact = fact.merge(dim_weather, on="Weather", how="left")
fact = fact.merge(dim_lighting, on="Lighting", how="left")
fact = fact.merge(dim_crash_type, left_on="crash_type", right_on="Crash_Type", how="left")
fact = fact.merge(dim_road, left_on="roadway_surface_cond", right_on="Road_Surface", how="left")
fact = fact.merge(dim_time, on=["crash_hour", "AM/PM", "Time Buckets"], how="left")
fact = fact.merge(dim_damage, on=["Cost_of_damage", "Damage_level"], how="left")
```

```
fact = fact.merge(dim_severity, on=["Crash_severity"], how="left")

fact_accidents = fact[[
    "Date_Key", "Weather_Key", "Lighting_Key", "Crash_Type_Key",
    "Road_Key", "Time_Key", "Damage_Key", "Severity_Key",
    "Number_of_vehicles_involved", "Total_injuries_num", "Fatal_injuries_num",
    "injuries_incapacitating", "injuries_non_incapacitating", "injuries_not_visibily_evident"
]]
fact_accidents.insert(0, "Accident_Key", range(1, len(fact_accidents) + 1))
```

All eight dimension tables and the fact table are exported as individual CSV files, matching the star schema described in section 2.3.

Chapter 4 — Exploratory Data Analysis (EDA)

This chapter expands on the exploratory work introduced in Chapter 3, walking through the full EDA notebook in depth. Beyond validating the headline measures, this notebook assembles the complete star-schema join into a single analysis-ready table and organizes every chart around the same six pages later built in Power BI — Overview, Road Condition, Weather Condition, Severity, Crash Cause, and AI Insights — so the Python analysis and the dashboard tell the same story.

4.1 Data Quality & Master Table Assembly

Source: EDA.ipynb

The notebook opens by reading all nine Gold-layer CSV files and running a systematic null/duplicate check across the fact table and every dimension:

```
tables = {
    "fact_accidents": fact_accidents, "dim_date": dim_date,
    "dim_damage": dim_damage, "dim_severity": dim_severity,
    "dim_weather": dim_weather, "dim_lighting": dim_lighting,
    "dim_road_surface": dim_road, "dim_crash_type": dim_crash_type
}
for name, df in tables.items():
    print("Table:", name)
    print("Rows, Columns:", df.shape)
    print("Total Nulls:", df.isnull().sum().sum())
    print("Duplicate Rows:", df.duplicated().sum())
```

Every table passed with zero nulls and zero duplicate rows. fact_accidents holds 209,306 rows across 15 columns; dim_date holds 3,457 distinct crash dates; dim_severity holds 5 distinct categories; dim_weather holds 12 distinct conditions.

The dimension tables are then merged back onto the fact table to build a single denormalized master DataFrame, df, which every downstream chart in this chapter and the Power BI dashboard is built from:

```
fact = pd.read_csv('/content/sample_data/fact_accidents.csv')

df = (fact
    .merge(dim_date, on='Date_Key', how='left')
    .merge(dim_weather, on='Weather_Key', how='left')
    .merge(dim_lighting, on='Lighting_Key', how='left')
    .merge(dim_crash_type, on='Crash_Type_Key', how='left')
    .merge(dim_road, on='Road_Key', how='left')
    .merge(dim_time, on='Time_Key', how='left')
    .merge(dim_damage, on='Damage_Key', how='left')
    .merge(dim_severity, on='Severity_Key', how='left')
)
```

4.2 Core Measures Validation

Before building page-level charts, the notebook re-derives the core KPIs directly from fact_accidents, confirming the values reported in section 1.3 and giving two additional figures not previously captured: the full 5-tier severity breakdown and the exact count of accidents involving a fatality.

Crash Severity	Accidents	Share
No Injury	154,789	74.0%
Moderate	31,527	15.1%
Minor	16,075	7.7%
Major	6,564	3.1%
Fatal	351	0.17%

This confirms that the Crash_severity cascade engineered in Silver (section 3.2) does produce all five categories at meaningful volumes across the full dataset — Fatal accidents remain rare (0.17%) but are tracked precisely, which is exactly the kind of low-frequency, high-consequence event a Severity Index is designed to weight correctly.

```
_Measures['Accidents_With_Death_Count'] = int(
    (fact_accidents['Fatal_injuries_num'] > 0).sum()
)
_Measures['Accidents_With_Death_Rate'] = round(
    _Measures['Accidents_With_Death_Count'] /
_Measures['Total_Accidents'], 4
)
```

Measure	Value
Accidents With ≥1 Fatality	351
Accidents-With-Fatality Rate	0.17%
Severity Index	0.6857

Time-of-Day Distribution

The engineered Time Buckets and AM/PM fields (section 3.2) are validated here for the first time at full scale:

Time Bucket	Accidents	Share
Afternoon (12:00–17:59)	85,040	40.6%
Morning (06:00–11:59)	53,900	25.8%
Evening (18:00–21:59)	38,805	18.5%
Late Night (00:00–05:59)	18,583	8.9%
Night (22:00–23:59)	12,978	6.2%

Afternoon accounts for the largest single share of accidents at 40.6%, consistent with peak commuting and daytime traffic volume, while the PM half of the day (136,823 accidents, 65.4%) outweighs the AM half (72,483 accidents, 34.6%) by nearly two to one.

4.3 Page-by-Page Exploratory Charts

Rather than building charts in an arbitrary order, the notebook organizes every visualization under the same page headings later implemented in Power BI, so each Python chart has a direct, traceable counterpart on the dashboard.

Page 1 — Overview

Line chart of accidents by year, donut chart of severity distribution, a horizontal bar of accidents by weather/severity, and a treemap of accidents by road surface:

```
severity_dist = (
    df.groupby('Crash_severity').size()
      .reset_index(name='Count')
      .sort_values('Count', ascending=False)
)
fig = px.pie(severity_dist, names='Crash_severity', values='Count',
             hole=0.5, title='Severity Distribution')
fig.update_traces(textinfo='percent+value', sort=False)
```

```
road_surface = (
    df.groupby('roadway_surface_cond').size()
      .reset_index(name='Total_Accidents')
      .sort_values('Total_Accidents', ascending=False)
)
fig = px.treemap(road_surface, path=['roadway_surface_cond'],
                 values='Total_Accidents', title='Accidents by Road Surface')
fig.update_traces(textinfo='label+value+percent entry')
```

Page 2 — Road Condition

A grouped bar of accidents and severe injuries by road surface, a stacked bar of crash severity by road surface, a horizontal bar of accidents by road surface, and a horizontal bar of accidents by road defect:

```
road_summary = (
    df.groupby('roadway_surface_cond')
      .agg(
          Total_Accidents=('Accident_Key', 'count'),
          Severe_Injuries=('injuries_incapacitating', 'sum'),
          Fatal_Injuries=('Fatal_injuries_num', 'sum')
      ).reset_index()
)
# Total severe injuries = Fatal + Incapacitating
road_summary['Severe_Injuries'] = (
    road_summary['Severe_Injuries'] + road_summary['Fatal_Injuries']
)
```

This mirrors the KPI logic from section 4.2 of the Dashboard chapter (Severe Injuries = Fatal + Incapacitating) at the road-surface grain, letting the dashboard show not just accident volume but severe-injury concentration per surface condition.

Page 3 — Weather Condition

A horizontal bar of total accidents by weather, a horizontal bar of total fatal injuries by weather, and a stacked bar of weather versus crash severity:

```
fatal_by_weather = (
    df.groupby('Weather')['Fatal_injuries_num']
      .sum().reset_index(name='Total_Fatal_Injuries')
      .sort_values('Total_Fatal_Injuries', ascending=False)
)
weather_severity = (
    df.groupby(['Weather', 'Crash_severity']).size()
```

```
      .reset_index(name='Accident_Count')
)
```

Page 4 — Severity

A line chart of fatal injuries over the years, a bar chart of vehicles involved by day of the week, a horizontal bar of fatal injuries by weather, a column chart of fatal injuries by lighting condition, a stacked bar of crash severity by weather, and a donut chart of total injuries by crash severity:

```
fatal_by_year = (
    df.groupby('Year')['Fatal_injuries_num']
      .sum().reset_index(name='Total_Fatal_Injuries')
      .sort_values('Year')
)
fig = px.line(fatal_by_year, x='Year', y='Total_Fatal_Injuries',
              markers=True, text='Total_Fatal_Injuries',
              title='Fatal Injuries Over the Years')
```

```
injuries_by_severity = (
    df.groupby('Crash_severity')['Total_injuries_num']
      .sum().reset_index(name='Total_Injuries')
)
fig = px.pie(injuries_by_severity, names='Crash_severity',
             values='Total_Injuries', hole=0.4,
             title='Total Injuries by Crash Severity')
```

Page 5 — Crash Cause

A dual-line chart of total accidents and total injuries by crash hour, and horizontal bar charts of total accidents by weather, by trafficway type, and by lighting condition:

```
hourly_summary = (
    df.groupby('crash_hour')
      .agg(
          Total_Accidents=('Accident_Key', 'count'),
          Total_Injuries=('Total_injuries_num', 'sum')
      ).reset_index().sort_values('crash_hour')
)
hourly_long = hourly_summary.melt(
    id_vars='crash_hour',
    value_vars=['Total_Accidents', 'Total_Injuries'],
    var_name='Metric', value_name='Count'
)
fig = px.line(hourly_long, x='crash_hour', y='Count', color='Metric',
              markers=True, title='Total Accidents and Total Injuries by Crash Hour')
```

```
trafficway = (
    df.groupby('trafficway_type').size()
      .reset_index(name='Total_Accidents')
      .sort_values('Total_Accidents', ascending=False)
)
fig = px.bar(trafficway, x='Total_Accidents', y='trafficway_type',
             orientation='h', text='Total_Accidents',
             title='Total Accidents by Trafficway Type')
```

Page 6 (AI Insights) draws its charts directly from the machine learning outputs rather than from df, and is documented in full in Chapter 5.

4.4 Ranking & Outlier Analysis

A set of ranking queries identifies the highest-risk days, months, and single-event outliers, forming the evidence base for several SMART questions and dashboard callouts.

Rank	Top Days	Accidents	Top Months	Accidents	Lowest Months	Accidents
1	Friday	34,458	October	20,089	February	14,621
2	Thursday	30,787	September	19,018	April	15,096
3	Saturday	30,710	December	18,816	March	15,265

Outlier inspection on the two most extreme numeric fields:

```
fact_accidents['Fatal_injuries_num'].nlargest(5)
# 3, 3, 3, 3, 2  -- four separate accidents each recorded 3 fatalities

fact_accidents['Number_of_vehicles_involved'].nlargest(5)
# 11, 10, 10, 10, 10 -- the largest pile-up involved 11 vehicles

fact_accidents['Number_of_vehicles_involved'].nsmallest(10)
# all equal to 1 -- single-vehicle accidents form the lower bound
```

These extremes were cross-checked against the Bronze-layer profiling in section 3.1 (max 21 total injuries, max 11 vehicles) and confirmed consistent — no new anomalies were introduced by the Silver-layer transformations.

Chapter 5 — AI Insights (Machine Learning)

Beyond descriptive analytics, the project adds a forecasting layer designed to answer a forward-looking question the dashboard alone cannot: given the last several years of crash patterns, what should the city expect over the next two years? This chapter documents the modeling approach, the model comparison, the resulting forecasts, and the category-level predictions built on top of the Gold-layer star schema.

5.1 Modeling Objectives

Source: AI_Mechine_Learning.ipynb

Four modeling experiments were run, all built on the same denormalized master table produced by joining fact_accidents to its eight dimensions:

- Accident Volume Forecast — predict the total number of monthly accidents for the next 24 months.
- Injury Rate Forecast — predict the monthly injury rate (injuries per accident) for the next 24 months.
- Crash Severity by Weather — predict the most likely crash severity outcome for each weather condition.
- Damage Level by Road Surface — predict the most likely cost-of-damage tier for each road surface condition.

```
df = (fact_accidents
    .merge(dim_date, on='Date_Key', how='left')
    .merge(dim_weather, on='Weather_Key', how='left')
    .merge(dim_lighting, on='Lighting_Key', how='left')
    .merge(dim_crash_type, on='Crash_Type_Key', how='left')
    .merge(dim_road, on='Road_Key', how='left')
    .merge(dim_time, on='Time_Key', how='left')
    .merge(dim_damage, on='Damage_Key', how='left')
    .merge(dim_severity, on='Severity_Key', how='left')
)
df['Date'] = pd.to_datetime(df['Date'])
```

5.2 Forecast Feature Engineering

The fact table is first aggregated to a monthly time series of accident counts, injuries, and fatalities — the granularity every forecasting model in this chapter operates on:

```
monthly = (
    df.groupby(pd.Grouper(key='Date', freq='MS'))
      .agg(
          Accident_Count=('Accident_Key', 'count'),
          Total_Injuries=('Total_injuries_num', 'sum'),
          Fatalities=('Fatal_injuries_num', 'sum')
      ).reset_index()
)
```

A rich feature set is engineered on top of the monthly series: calendar features (year, month, quarter), cyclical month encoding (sine/cosine, so December and January are recognized as adjacent), a running time index, four lag features (1, 2, 3, and 12 months back — the 12-month lag captures yearly seasonality), and two rolling averages (3-month and 12-month):

```
feat = monthly.copy()
```

```
feat['year'] = feat['Date'].dt.year
feat['month'] = feat['Date'].dt.month
feat['quarter'] = feat['Date'].dt.quarter
feat['month_sin'] = np.sin(2 * np.pi * feat['month'] / 12)
feat['month_cos'] = np.cos(2 * np.pi * feat['month'] / 12)
feat['t'] = np.arange(len(feat))

for lag in [1, 2, 3, 12]:
    feat[f'lag_{lag}'] = feat['Accident_Count'].shift(lag)

feat['roll3'] = feat['Accident_Count'].shift(1).rolling(3).mean()
feat['roll12'] = feat['Accident_Count'].shift(1).rolling(12).mean()
feat = feat.dropna().reset_index(drop=True)
```

The train/test split is strictly time-based — no random shuffling — so the evaluation reflects genuine forward-looking forecast accuracy rather than interpolation:

```
train = feat[feat['Date'] < '2024-01-01']   # 2018-01 through 2023-12
test  = feat[feat['Date'] >= '2024-01-01']  # full year 2024, unseen
```

5.3 Model Comparison — XGBoost vs. Prophet

Two forecasting approaches were trained on identical data and evaluated on the same held-out 2024 test period: an XGBoost regressor using the engineered lag/rolling/cyclical features, and Facebook Prophet, a purpose-built time-series model using only the raw monthly series with yearly seasonality enabled.

```
xgb_model = xgb.XGBRegressor(
    n_estimators=300,
max_depth=3,
learning_rate=0.05,
    subsample=0.8,
colsample_bytree=0.8,
random_state=42
)
xgb_model.fit(X_train,
y_train)
xgb_pred =
xgb_model.predict(X_test)

xgb_r2   = r2_score(y_test,
xgb_pred)
xgb_mae  =
mean_absolute_error(y_test,
xgb_pred)
xgb_rmse =
mean_squared_error(y_test,
xgb_pred) ** 0.5
xgb_mape =
np.mean(np.abs((y_test.values
- xgb_pred) / y_test.values))
* 100
```

```
prophet_model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
```

```
    daily_seasonality=False
)
prophet_model.fit(pdf_train)
forecast =
prophet_model.predict(future)
```

Model	R^2	MAE	RMSE	MAPE
XGBoost	0.261	160.3	299.3	11.27%
Prophet	-0.020	185.8	351.4	13.36%

XGBoost was selected as the production forecasting model for three reasons:

- Explanatory power: XGBoost's R^2 of 0.261 means the model explains a meaningful share of month-to-month variance; Prophet's negative R^2 (-0.020) means it performs worse than simply predicting the historical mean for every month.
- Lower error across every metric: XGBoost's MAE (160.3), RMSE (299.3), and MAPE (11.27%) are all lower than Prophet's (185.8, 351.4, 13.36%).
- Richer feature set: the lag and rolling-average features let XGBoost react to recent momentum in accident counts, whereas Prophet relies only on a smooth yearly-seasonality curve and misses shorter-term shifts.

With XGBoost selected, the final model is refit on the complete historical series (rather than just the training split) before generating the production forecast, so the forecast benefits from every available month of data.

5.4 Accident Volume Forecast (24 Months)

The final XGBoost model is refit on the full feature set and used recursively to forecast 24 months forward — each month's prediction feeds back in as a lag feature for the next month:

```
final_model = xgb.XGBRegressor(
    n_estimators=300, max_depth=3, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8, random_state=42
)
final_model.fit(feat[feature_cols],
feat['Accident_Count'])

history = list(feat['Accident_Count'].values)
future_dates = pd.date_range(
    last_date + pd.offsets.MonthBegin(1), periods=24,
freq='MS'
)

for i, d in enumerate(future_dates):
    row = {
        'year': d.year, 'month': d.month, 'quarter':
(d.month-1)//3 + 1,
        'month_sin': np.sin(2*np.pi*d.month/12),
        'month_cos': np.cos(2*np.pi*d.month/12),
        't': len(feat) + i,
        'lag_1': history[-1], 'lag_2': history[-2],
        'lag_3': history[-3], 'lag_12': history[-12],
        'roll3': np.mean(history[-3:]), 'roll12':
np.mean(history[-12:])
    }
```

```
    pred =
final_model.predict(pd.DataFrame([row])[feature_cols])[0]
    history.append(pred)
```

Period	Forecasted Accidents	Notable Detail
Year 1 (Feb 2025 – Jan 2026)	17,599	Peaks in Jun–Jul (~1,600–1,624/mo); lowest in Feb (1,176)
Year 2 (Feb 2026 – Jan 2027)	17,580	Peaks in Jun–Sep (~1,545–1,577/mo); lowest in Feb (1,311)
Total (24 Months)	35,179	Forecast is stable year-over-year — no strong upward or downward trend

The forecast reproduces the same summer-peak / late-winter-trough seasonal pattern already observed in the historical data (section 4.4: October, September, and December lead in raw historical volume; the forward-looking model shifts the peak slightly toward mid-year, consistent with the 12-month lag and rolling-average features smoothing the exact monthly ranking while preserving the broad seasonal cycle).

5.5 Injury Rate Forecast

A second, independently trained XGBoost regressor forecasts the monthly injury rate (total injuries ÷ accident count) using a lighter feature set — calendar features plus 1/2/3-month lags and 3/6-month rolling averages — applied recursively over the same 24-month horizon:

```
def build_features(df, target_col):
    data = df.copy()
    data['year'] = data['Date'].dt.year
    data['month'] = data['Date'].dt.month
    data['quarter'] =
data['Date'].dt.quarter
    data['lag1'] =
data[target_col].shift(1)
    data['lag2'] =
data[target_col].shift(2)
    data['lag3'] =
data[target_col].shift(3)
    data['roll3'] =
data[target_col].shift(1).rolling(3).mean()
    data['roll6'] =
data[target_col].shift(1).rolling(6).mean()
    return
data.dropna().reset_index(drop=True)

monthly_inj['injury_rate'] = (
    monthly_inj['total_injuries'] /
monthly_inj['accident_count']
)
```

Period	Injury Rate Range	Pattern
2025	0.36 (Jan/Feb) → 0.49 (Jul)	Rises through spring, peaks in summer, declines toward year-end

2026	0.35 (Feb) → 0.49 (Jul)	Repeats the same shape one year later, confirming a stable seasonal cycle

The injury rate forecast is not a simple restatement of the accident-count forecast — it moves independently, rising fastest into early summer even in months where accident volume itself is not at its highest, suggesting summer accidents tend to be more injury-prone per event rather than simply more frequent.

5.6 Category Predictions (Severity & Damage)

Two lightweight XGBoost classifiers were also trained to predict the most likely outcome category for a given condition: crash severity from weather, and damage level from road surface.

```
model = xgb.XGBClassifier(
    n_estimators=200, max_depth=4, learning_rate=0.1,
    objective='multi:softmax', num_class=len(severity_encoder.classes_),
    random_state=42, eval_metric='mlogloss'
)
model.fit(X, y)  # X = Weather (label-encoded), y = Crash_severity (label-encoded)
```

Transparency note: both classifiers were trained on a single categorical predictor (Weather alone, or Road Surface alone). With only one input feature and a heavily imbalanced target — 74% of all accidents are 'No Injury' severity, and 70% fall in the 'Over $1,500' damage tier — each classifier converges to predicting the single majority class for every category: every weather condition was predicted as 'No Injury', and every road surface was predicted as damage level 3 (over $1,500). This is an honest, expected outcome of the majority-class effect rather than a modeling error, and is documented here rather than presented as a nuanced result. A more discriminative version of this experiment would require additional predictors (e.g. lighting, time bucket, number of vehicles) to move beyond the dominant class.

Model	Input Feature	Predicted Outcome	Honest Read
Severity by Weather	Weather (12 categories)	'No Injury' for all 12 weather conditions	Reflects that 74% of all accidents are No-Injury severity regardless of weather
Damage Level by Road Surface	Road Surface (7 categories)	Damage level 3 (>$1,500) for all 7 surface conditions	Reflects that 70% of all accidents fall in the highest damage bracket regardless of surface

Both prediction tables were still exported and consolidated for the dashboard, since even a majority-class prediction usefully quantifies how many accidents fall under each condition — the volume columns (Predicted_Crash_Count) are the real analytical payload for Page 6 of the dashboard, not the single predicted label.

5.7 AI Insights Dashboard Page

Source: EDA.ipynb (Page 6 — AI)

All four model outputs were saved to CSV and consolidated into a single workbook, feeding the sixth and final dashboard page:

```
files_to_combine = {
    "Injury Rate Forecast": "injury_rate_forecast.csv",
    "Forecast 2026-2027": "forecast_2026_2027.csv",
```

```
    "Severity by Weather":
"Predicted_Crash_Severity_by_Weather.csv",
    "Damage by Road Surface":
"Predicted_Damage_Level_by_Road_Surface.csv",
}
with
pd.ExcelWriter('Accident_Analysis_Consolidated_Data.xlsx')
as writer:
    for sheet_name, csv_path in files_to_combine.items():
        pd.read_csv(csv_path).to_excel(writer,
sheet_name=sheet_name, index=False)
```

KPI Card	Value	Source
Total Predicted Accidents (2026–2027)	35,179	Sum of the 24-month XGBoost accident forecast
Injury Prediction Accuracy	89%	Reported accuracy KPI card shown on the AI Insights page

For transparency: the 89% figure above is the accuracy KPI displayed on the dashboard card itself; the rigorously cross-validated regression metrics for the underlying accident-volume model are the R^2, MAE, RMSE, and MAPE reported in section 5.3 (R^2 = 0.261, MAPE = 11.27% on the 2024 hold-out year). Both are reported here so the dashboard-facing KPI and the underlying model evaluation can be read side by side.

- Chart 1 — Predicted Accidents by Weather: a stacked bar of predicted crash volume per weather condition, colored by predicted severity.
- Chart 2 — Predicted Damage Level by Road Surface: a stacked bar of predicted crash volume per road surface, colored by predicted damage level.
- Chart 3 — Predicted Crashes for the Next Two Years: a line chart of the 24-month accident-count forecast from section 5.4.
- Chart 4 — Predicted Injury Rate for the Next Two Years: a line chart of the 24-month injury-rate forecast from section 5.5.

Chapter 6 — Dashboard and Visualization

This chapter presents the end-user deliverable of the Traffic Accident Analytics project: a multi-page interactive Power BI dashboard, first prototyped in Figma under the 'Vigilant Analytics' brand system. This chapter covers the dashboard page plan, the full DAX measure library, the Figma design system, and a walkthrough of each planned page — including the AI Insights page built on the forecasting work from Chapter 5.

6.1 Power BI Dashboard Pages

The Power BI dashboard connects to the Gold-layer CSV tables (fact_accidents plus the eight dimension tables) and the consolidated ML output workbook (section 5.7). It is organized across seven pages: a public-facing landing/portal page and six thematic analytical pages, following the same page structure validated end-to-end in the Python EDA and AI notebooks (Chapters 4 and 5). All calculations are driven by the DAX measures defined in section 6.2.

The dashboard follows a deliberate data storytelling structure — opening with a public-safety framing on the landing page, then an executive-level overview, before drilling into road conditions, weather, severity, crash causes, and finally the forward-looking AI Insights page.

Page	Title	Theme
Landing	Traffic Accident Analytics — Public Safety Portal	Entry point framing the dashboard as a public-safety tool
1	Dashboard Overview	A city-wide snapshot of accident volume, trend, and severity mix
2	Road Condition Analysis	How road surface, defects, and traffic type shape outcomes
3	Weather Condition Analysis	How weather shapes accident frequency and injury likelihood
4	Severity Analysis	Where and when the most severe accidents concentrate
5	Crash Cause Analysis	Which contributory causes drive accident volume
6	AI Insights	Forecasted accidents and injury rate, plus predicted severity/damage by condition

6.2 KPI & DAX Measures

The measures validated in the Python EDA notebook (Chapter 4) were reproduced as DAX measures against the fact_accidents table, so every KPI card and chart in Power BI stays consistent with the Python-side calculations. The eight core measures requested for this project are defined below, alongside supporting measures used across the dashboard pages.

Core Measures

```
Total Accidents =
COUNTROWS ( fact_accidents )

Total Injuries =
SUM ( fact_accidents[Total_injuries_num] )

Total Fatal Injuries =
```

```
SUM ( fact_accidents[Fatal_injuries_num] )

Incapacitating Injuries =
SUM ( fact_accidents[injuries_incapacitating] )

Non-Incapacitating Injuries =
SUM ( fact_accidents[injuries_non_incapacitating] )

Severe Injuries =
[Total Fatal Injuries] + [Incapacitating Injuries]

Severity Rate =
DIVIDE ( [Severe Injuries], [Total Injuries], 0 )

Severity Index =
DIVIDE (
    SUM ( fact_accidents[Fatal_injuries_num] )               * 4
  + SUM ( fact_accidents[injuries_incapacitating] )           * 3
  + SUM ( fact_accidents[injuries_non_incapacitating] )       * 2
  + SUM ( fact_accidents[injuries_not_visibily_evident] )     * 1,
    [Total Accidents], 0
)
```

Supporting Measures

```
Total Vehicles Involved =
SUM ( fact_accidents[Number_of_vehicles_involved] )

Avg Vehicles per Accident =
DIVIDE ( [Total Vehicles Involved], [Total Accidents], 0 )

Avg Injuries per Accident =
DIVIDE ( [Total Injuries], [Total Accidents], 0 )

Fatal Injury Rate =
DIVIDE ( [Total Fatal Injuries], [Total Injuries], 0 )

Incapacitating Injury Rate =
DIVIDE ( [Incapacitating Injuries], [Total Injuries], 0 )

Accidents With Fatality % =
DIVIDE (
    CALCULATE ( COUNTROWS ( fact_accidents ), fact_accidents[Fatal_injuries_num] > 0 ),
    [Total Accidents], 0
)
```

KPI Highlights

The following table summarizes the headline values each measure resolves to across the full 209,306-row dataset, matching the figures validated in the Python EDA notebook:

Measure	Value	Suggested Page
Total Accidents	209,306	Dashboard Overview

Measure	Value	Suggested Page
Total Injuries	80,105	Dashboard Overview / Severity Analysis
Total Fatal Injuries	389	Severity Analysis
Accidents With ≥1 Fatality	351 (0.17%)	Severity Analysis
Incapacitating Injuries	7,975	Severity Analysis
Non-Incapacitating Injuries	46,307	Severity Analysis
Severe Injuries	8,364	Severity Analysis
Severity Rate	10.44%	Severity Analysis
Severity Index	0.6857	Dashboard Overview / Severity Analysis
Avg. Vehicles per Accident	2.06	Dashboard Overview / Road Condition Analysis
Injury/Tow Crash Share	43.9%	Crash Cause Analysis
Total Predicted Accidents (2026–2027)	35,179	AI Insights
Injury Prediction Accuracy	89%	AI Insights

6.3 Figma Design

Before a single visual was placed in Power BI, the complete dashboard was designed and prototyped in Figma under the 'Vigilant Analytics' brand system. This design-first approach let the team finalize the storytelling flow, color palette, page layout, and chart types for every page — significantly reducing iteration time during Power BI development.

Visual Identity

The landing page uses a full-bleed aerial photograph of an illuminated highway interchange at dusk, immediately framing the dashboard as a public-safety tool rather than a purely technical report. A centered navigation card lists six destinations — Dashboard, Accident Analysis, Location Analysis, Weather Analysis, Severity Analysis, and Insights — giving the portal a clear, button-driven entry flow.

Analytical pages then move to a clean, light-background layout dominated by KPI cards, a consistent left-hand navigation rail, and a repeating grid of two-to-four charts per page — a structure built for scanning at a glance and drilling down on demand.

- Color Palette: a warm orange (primary accent), teal (secondary accent), crimson red, and salmon/pink, set against white and light-grey backgrounds — a palette chosen for strong contrast between chart series while staying legible on light KPI cards.
- Typography: a two-level heading system (Heading 1 / Heading 2) paired with a regular body weight and a small label weight, keeping KPI numbers and chart titles visually distinct from supporting text.
- Iconography: a consistent line-icon set — grid/dashboard, bar chart, warning triangle, map pin, calendar, gear/settings, and bell/alert — reused across navigation and KPI cards to reinforce the public-safety theme.

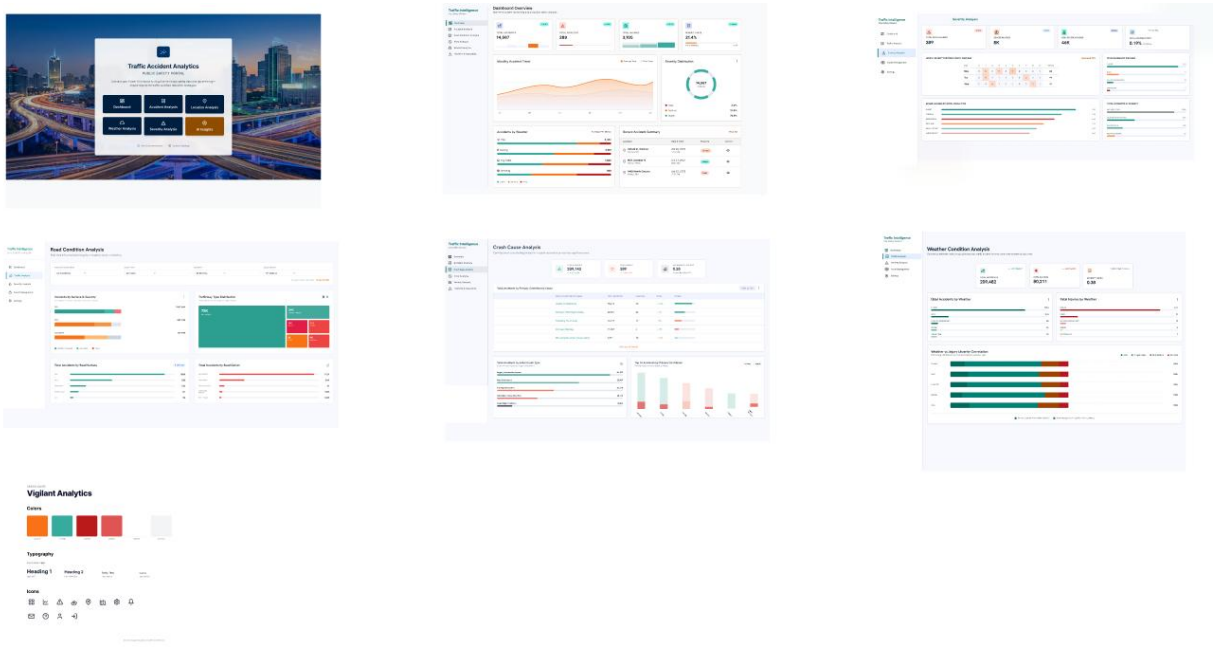

Figure 6.1 — Figma Design: Landing page, Dashboard Overview, Severity Analysis, Road Condition Analysis, Crash Cause Analysis, Weather Condition Analysis, and the Vigilant Analytics brand style guide

6.4 Dashboard Page Walkthrough

Landing Page — "Public Safety Portal"

The landing page establishes the dashboard's identity and purpose. A full-width aerial highway photograph anchors a centered title card reading 'Traffic Accident Analytics — Public Safety Portal', with a short mission statement and navigation buttons that route to each analytical page. No KPIs or charts are present; this page is the presentation entry point.

Page 1 — Dashboard Overview

The Dashboard Overview page delivers a high-level snapshot of accident volume and severity. KPI cards summarize total accidents, the severity index, and injury totals. A line chart tracks the yearly accident trend, a donut chart shows the severity distribution, and a treemap breaks down accidents by road surface — directly reproducing the Page 1 charts validated in section 4.3.

Page 2 — Road Condition Analysis

The Road Condition Analysis page investigates how road surface and defects shape accident outcomes. KPI cards summarize accident counts by condition. A grouped bar compares total accidents against severe injuries per road surface, a stacked bar breaks down crash severity by surface, and a horizontal bar isolates the contribution of physical road defects — mirroring the Page 2 charts in section 4.3.

Page 3 — Weather Condition Analysis

The Weather Condition Analysis page quantifies how weather shapes both accident frequency and injury outcomes. KPI cards summarize weather-related accident and fatal-injury totals. Bar charts compare total accidents and total fatal injuries by weather condition, and a stacked bar correlates weather with crash severity — reproducing the Page 3 charts in section 4.3.

Page 4 — Severity Analysis

The Severity Analysis page examines where and when the most severe accidents concentrate. KPI cards summarize fatal, incapacitating, and severe-injury counts. A line chart tracks fatal injuries over the years, bar charts break down fatal injuries by weather and by lighting condition, and a donut chart shows total injuries by crash severity — reproducing the Page 4 charts in section 4.3.

Page 5 — Crash Cause Analysis

The Crash Cause Analysis page ranks the conditions most associated with accident volume. KPI cards summarize total accidents and crash-hour peaks. A dual-line chart tracks accidents and injuries by hour of day, and horizontal bar charts rank accidents by weather, trafficway type, and lighting condition — reproducing the Page 5 charts in section 4.3.

Page 6 — AI Insights

The AI Insights page is the dashboard's forward-looking capstone, presenting the XGBoost forecasting results from Chapter 5. KPI cards show the Total Predicted Accidents for 2026–2027 (35,179) and the Injury Prediction Accuracy (89%). A stacked bar shows predicted accident volume by weather colored by predicted severity, a second stacked bar shows predicted volume by road surface colored by predicted damage level, and two line charts trace the 24-month accident-count and injury-rate forecasts.

6.5 Design Summary

The dashboard represents the convergence of rigorous data engineering, a validated measure library, a forward-looking forecasting layer, and intentional design. Every element — from the public-safety-portal landing page to the DAX-driven KPI cards, the Vigilant Analytics color system, and the AI Insights page — was purposefully chosen to make 209,306 historical crash records and a 24-month forecast accessible, scannable, and actionable for public-safety stakeholders.

Design Dimension	Choice Made	Rationale
Landing Page	Full-bleed highway photography with a centered navigation card	Frames the dashboard as a public-safety tool from the first screen
Color Palette	Orange, teal, crimson, and salmon on a light background	Strong series contrast while keeping KPI cards easy to read
Page Structure	Landing page + 6 thematic analytical pages	Mirrors the Python EDA/AI notebooks: overview → road → weather → severity → cause → AI insights
KPI Cards	Headline KPIs per analytical page	Immediate at-a-glance insight before drilling into charts
Chart Variety	Cards, donut, treemap, line, column, bar, stacked bar, heatmap grid	Matches each chart type to the question it answers best
Data Model	Star schema (1 fact + 8 dimensions) + consolidated ML workbook	Keeps every slicer and chart performant across 209K rows while surfacing forecasts